



model-scanner

Static Security Scanner for AI Model Weights
Before Hugging Face → vLLM Handoff

A first-line defense that inspects model files without loading them —
catching hostile payloads, path escapes, and untrusted publishers
before they reach your AI infrastructure.

Version 0.4.1 · Python 3.9+ · Zero runtime dependencies

github.com/Elpi97/local-ai-model-security-scanner

The Problem

The attack surface: We download model weights from Hugging Face and hand them to vLLM for serving. Those weights are *opaque binary files* — pickle, safetensors, ONNX, GGUF. A hostile or compromised model file can execute arbitrary code the moment it's loaded. **There is no first-line check today.**

Why this matters

- **Pickle deserialization is a known RCE vector.** A crafted pickle can call `os.system()`, `subprocess.Popen()`, or any arbitrary function during `torch.load()` — the exact call vLLM makes.
- **Hugging Face has had malicious model incidents.** The platform is open — anyone can publish. Publisher reputation is not a guarantee.
- **Safetensors and ONNX have their own attack vectors:** path traversal via external-data references, zip-slip in PyTorch checkpoints, header manipulation in safetensors.
- **The attack fires at load time** — once vLLM calls `torch.load()` or `onnx.load()`, it's already too late.

The gap in existing tools

TOOL	WHAT IT DOES	WHY IT'S NOT ENOUGH
<code>weights_only=True</code>	PyTorch's restricted unpickler	Only blocks <i>some</i> pickle attacks. Doesn't cover safetensors, ONNX, or zip-slip. No audit trail.
Hugging Face scanning	Signature-based malware check on upload	Catches <i>known</i> malware. Misses novel payloads. No structural analysis.
ClamAV / YARA	Generic antivirus	Doesn't understand model file formats. Can't check publisher provenance or ONNX graph structure.
Manual review	Human inspects the file	Doesn't scale. No consistency guarantee. No audit record.

model-scanner fills this gap: structural static analysis + provenance verification + audit trail, with zero dependencies, running before the model reaches vLLM.

The Solution — Three Tiers

model-scanner is a static security scanner that runs **before** the model reaches vLLM. It **never loads the model** — it inspects the bytes, validates provenance, and produces an audit-trail report with a clear verdict.

TIER	WHAT IT CHECKS	ANALOGY
1 — File Safety CRITICAL REVIEW	Does the file contain hidden code, path escapes, or hostile structures? <i>Pickle opcode analysis, ONNX deep protobuf parse, safetensors header validation, GGUF magic check, PyTorch zip-slip detection.</i>	We don't open the package — we X-ray it.
2 — Trust & Integrity REVIEW SAFE	Who published it? Does the hash match? Is the publisher allowlisted? <i>SHA-256 verification, Hugging Face publisher allowlist, LFS digest comparison, manifest validation.</i>	Checking the return address and the seal.
3 — Behavior Checklist Manual	Does the model refuse harmful requests? <i>Deferred to the AI department's vLLM testbed. The scanner attaches a checklist to every report so the analyst signs off.</i>	Interviewing the candidate before hiring.

Verdicts

SAFE

All Tier 1 checks passed, publisher allowlisted (or review accepted), hash matches.

REVIEW

Something needs human attention — unallowlisted publisher, unusual structure, or shallow scan (deep parse unavailable). Not necessarily dangerous.

DANGEROUS

A concrete threat was found — path escape, dangerous code reference, hostile op domain.

Do not hand off.

Exit code 1 = DANGEROUS, 0 = SAFE/REVIEW. CI-integrable.

Key design principles

- **Never load the artifact being scanned.** All analysis is static — pickle opcode disassembly, protobuf parsing, header inspection. No `torch.load()`, no `onnx.load()` with execution.
- **Zero runtime dependencies.** Pure Python 3.9+ stdlib. The optional `[onnx]` extra enables deep ONNX parsing but isn't required.
- **Audit trail on every scan.** Markdown report with analyst sign-off table, JSON for CI/CD. Every finding is traceable to a file, a line, and a check.
- **Honest severity tiers.** The report explicitly states what was checked and what wasn't. No silent gaps.

Live Demo — 60 Seconds

Step 1: Install

```
$ curl -fsSL https://raw.githubusercontent.com/Elpi97/local-ai-model-security-scanner/main/install.sh
✓ model-scanner installed to ~/.local/share/model-scanner
✓ CLI available at ~/.local/bin/model-scanner

$ model-scanner --doctor
model-scanner 0.4.1 – install self-diagnosis
  Python:      3.11.x
  onnx:        present and working (1.22.0)
  deep scan:   ENABLED
✓ Your install is OK.
```

Step 2: Scan a real model (SAFE)

```
$ model-scanner gpt2/model.safetensors --hf-repo gpt2 --publisher openai
🟢 SAFE – gpt2/model.safetensors
Tier 1: safetensors header OK (64 tensors)
Tier 2: publisher "openai" allowlisted, SHA-256 verified
Tier 3: behavior checklist attached (6 items, manual)

Report: gpt2/model.safetensors.report.md
```

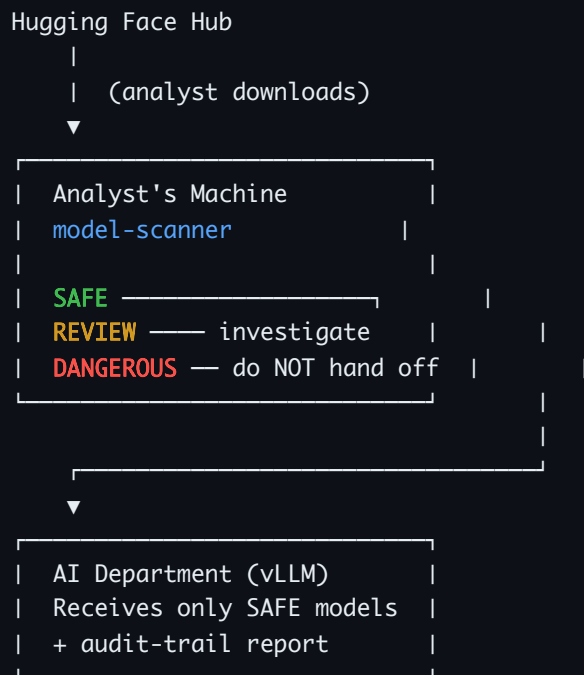
Step 3: Catch a hostile model (DANGEROUS)

```
$ model-scanner trojan.onnx
🔴 DANGEROUS – trojan.onnx
CRITICAL: ONNX external-data location '../../../../../etc/passwd'
           escapes model directory – path traversal attack.
CRITICAL: Non-standard op domain "evil.custom" – not in standard
           domains (ai.onnx, ai.onnx.ml, ai.onnx.preview.training).

Exit code: 1
Report: trojan.onnx.report.md (see audit trail)
```

What just happened: The scanner detected a path-traversal attack in the ONNX model's external-data reference — without loading the model. The `../../../../../../../../etc/passwd` path would have been followed by vLLM's ONNX loader, potentially leaking system files. The scan took **under 1 second**.

How It Fits the Workflow



Key integration points

- **Runs on the analyst's machine, not in the serving pipeline.** Zero latency impact on vLLM. The AI dept only receives files that passed the scan.
- **The audit-trail report travels with the model.** The AI dept can verify what was checked, by whom, and when. Sign-off table included in every Markdown report.
- **Exit codes are CI/CD-friendly.** Exit 1 = DANGEROUS (block), 0 = SAFE or REVIEW (proceed with awareness). Integrates with any pipeline that checks exit codes.
- **Drop-folder workflow.** The analyst scans, reviews the report, and moves SAFE files to the drop folder the AI dept picks up. No API integration required — works with existing file-based workflows.

What it catches

- Pickle RCE payloads (dangerous globals, `os.system` calls, `__reduce__` tricks)
- ONNX external-data path escapes (`./etc/passwd`, URLs, absolute paths)
- Non-standard ONNX op domains (custom malicious ops)
- PyTorch zip-slip (path traversal in checkpoint archives)
- Safetensors header manipulation
- GGUF format tampering
- Publisher spoofing, hash mismatches
- Oversized embedded initializers (potential data smuggling)

What it doesn't catch (yet)

- **Runtime behavioral issues** — the model *says* harmful things. Tier 3 is a manual checklist, not automated. This belongs in the AI dept's vLLM testbed.

- **Adversarial weights** — mathematically perturbed to misbehave. No static scanner can detect these; it's a research problem.
- **Novel attack vectors in future format versions** — the scanner tracks current format specs. New formats or versions may need new checks.

Why this honesty matters: Overstating a security tool's coverage is worse than understating it. Every report explicitly lists what was checked and what wasn't — no silent gaps.

Handling Likely Objections

OBJECTION	RESPONSE
"vLLM already uses <code>weights_only=True</code> "	That blocks <i>some</i> pickle attacks but not all (safetensors, ONNX, zip-slip are unaffected). Defense in depth — this catches it <i>before</i> it reaches the loader, with an audit record.
"Can't we just use Hugging Face's scanning?"	HF scans for <i>known</i> malware signatures. This does <i>structural static analysis</i> — it catches novel payloads HF hasn't seen yet. Different threat model.
"Will this slow down our pipeline?"	It runs in <1s per file on the analyst's machine, not in the serving path. The AI dept never sees it. Zero latency impact on vLLM.
"Why not just use ClamAV / YARA?"	Those are generic signature-based AV. This understands model file formats (pickle opcodes, ONNX protobuf, safetensors headers) and checks ML-specific threats (publisher provenance, external-data escapes, custom op domains).
"Who maintains it?"	Stdlib-only Python, 94 tests, zero runtime deps. Low maintenance burden. The optional <code>[onnx]</code> extra is from the official ONNX project.
"What if it flags a legitimate model?"	REVIEW is not DANGEROUS — it means "investigate." The analyst looks at the finding, decides, and signs off in the audit report. DANGEROUS is reserved for concrete threats.

Getting Started

```
# One-command install (creates isolated venv, installs with [onnx] extra)
curl -fsSL https://raw.githubusercontent.com/Elpi97/local-ai-model-security-scanner/main/install.sh

# Verify your install
model-scanner --doctor

# Scan a model file
model-scanner path/to/model.safetensors --hf-repo org/model-name --publisher org-name

# Scan with expected hash (Tier 2 integrity)
model-scanner model.onnx --expected-sha256 abc123...

# Allow a custom ONNX op domain
model-scanner model.onnx --allow-onnx-domain com.mycompany

# Generate a full audit-trail report
model-scanner model_dir/ --doc-report --publisher my-org --allowlist config/publishers.json
```

Resources

DOCUMENT	PURPOSE
README.md	What it is, installation, flags, quick start — the elevator pitch
HOW_TO_USE.md	Operator guide — all flags, all formats, examples
examples/DEMO.md	Live end-to-end demo with screenshot — proof it works on real models

